Massive Scale Data Analytics using Spark

Final Report

Arbaaz Jaffer

BSc Computer Science

Supervised By: Dr. Daniel O'Keeffe

Royal Holloway, University of London

Date: April 15, 2026

Contents

1 Abstract **3**

2 Introduction **4**
2.1 The Problem 4
2.2 Aims and Goals of the project 4
2.3 Extended Scope and Contributions 5
2.4 Planning and time-scale 5
2.5 Later-Phase Timeline and planning 6

3 Background Theory **7**
3.1 Big Data Characteristics and Challenges 7
3.2 Distributed Computing Models 7
3.2.1 MapReduce 7
3.2.2 HDFS and Cluster Architecture 8
3.3 Apache Spark Overview 8
3.3.1 Spark Architecture 8
3.3.2 RDDs, DataFrames, and Datasets 10
3.3.3 Transformations, Actions, and Lazy Evaluation 10
3.4 Performance Concepts 11
3.4.1 Parallelism, Partitions, Shuffles 11
3.4.2 Memory Management 12
3.4.3 Distributed Statistics and Approximation 12
3.4.4 Spark Submit Parameters 13
3.5 Columnar Storage and Analytical Query Performance 13
3.6 Bucketisation for Scalable Frequency Distributions 14
3.7 Machine Learning and Clustering 14
3.7.1 Unsupervised Learning and K-Means 14
3.7.2 Clustering Evaluation Metrics 15

4 Implementation and Experimental Work **16**
4.1 Architecture of the Spark Application 16
4.2 Interim Data Source 16
4.3 Final Data Source 16
4.4 Pre-processing and Cleaning Pipeline 17
4.5 Word Count Implementation (Problem 1) 17
4.6 Distributed Grep Implementation (Problem 2) 18
4.7 Experimental Design and Execution 19
4.7.1 Small-scale Tests 19
4.7.2 10 GB to 90 GB Tests 19
4.7.3 Full 100 GB Tests 19
4.8 Statistical Analysis Implementation (Problem 3) 20
4.9 K-Means Clustering Implementation (Problem 4) 21
4.9.1 Output Management and Post-Processing Workflow 24

5 Performance Analysis **25**
5.1 Experimental Setup . . . 25
5.2 Spark-submit Parameter Tuning . . . 25
5.2.1 Memory Settings . . . 25
5.2.2 Cores and Executors . . . 26
5.3 Scalability Benchmarks . . . 28
5.4 Interim Recommendations . . . 28
5.5 Later-Phase Performance Analysis . . . 29
5.5.1 Statistical Analysis Performance . . . 29
5.5.2 K-Means Performance . . . 31
5.6 Combined Recommendations . . . 32

6 Critical Analysis and Discussion **33**
6.1 Comparing Workload Types . . . 33
6.2 Quality of Results . . . 33
6.3 Limitations . . . 34

7 Software Engineering **35**
7.1 Methodology . . . 35
7.1.1 Iterative Development . . . 35
7.1.2 Testing Methodology . . . 35
7.1.3 Object-Oriented Design . . . 36
7.2 System Design . . . 36

1 Abstract

The rapid growth of data volume and complexity has raised the need to understand what exactly big data is, how to process it efficiently, and the challenges that come along with it. This also brings about the need for distributed computing frameworks and clusters of machines capable of handling such data as conventional systems using single computers would be obsolete in this context due to the sheer scale of data involved.

This leads to the key tool used within the project, Apache Spark, a distributed computing framework that has allowed for large scale data processing across clusters of computers in a fault-tolerant and extremely efficient manner. Some of the key features of Spark processes include in-memory computation, which drastically reduces the time it takes to process data compared to conventional methods. Additionally, Apache Spark supports the ability to process data via batch processing, which is useful for large datasets, as well as stream processing, which allows for real-time data analytics.

For this project, the aim is to explore the capabilities of Apache Spark, using the PySpark library to implement two programs that can handle large scale datasets. The first program will be a word count application that counts the frequency of each word in a large text file that would need to be cleaned and preprocessed before the word count can be performed. The second program will be a distributed grep application that searches for a key word or pattern in the same large text-based dataset and returns all titles containing that keyword or pattern.

The project was then extended beyond text processing to go on to explore structured numerical analytics and unsupervised machine learning. A statistical analysis program was implemented over the NOAA Global Historical daily weather dataset, also approximately 100 GB in size, computing descriptive statistics and a frequency distribution using distributed approximation methods created for large scale computation. A K-Means cluster program was also made using Spark MLlib to group weather stations into temperature based zones, with the result visualised on a global map and scatter plot to confirm that the results had produced a geographically coherent set of clusters.

Performance analysis will be conducted to evaluate the efficiency and scalability of the programs within this project by running them on the university's cluster and tuning various Spark-submit parameters such as executor memory, the number of cores and partitions of data. The results of these experiments will be displayed and discussed in detail, highlighting the strengths and weaknesses of the programs created, and providing an insight into the performance of Apache Spark for large scale data processing tasks. These results will also be used to provide recommendations for future workloads and optimisation strategies when using Apache Spark for big data analytics.

Overall, this project provides a deeper understanding of PySpark and its capabilities for massive scale data processing, using clusters of computers in parallel to handle large quantities of data efficiently producing conclusions and recommended configuration settings.

2 Introduction

2.1 The Problem

Around 90 percent of the world's data has been generated in the last two years alone [1]. This rate of data generation is expected to continue growing exponentially, with estimates suggesting that over 4 years, the total amount of data created globally will continue to double [1]. This explosion of data has led to the emergence of the term "big data", which refers to these massive datasets that are too large and complex to be processed using traditional data processing methods [3]. The complexity and quantity of the data being generated will only continue to increase, making it essential to develop efficient methods for processing and analysing this data. Traditional data processing methods, which involve using a single machine to process the data, are no longer sufficient in the time taken and scalable enough to handle this increasing volume of data.

This has led to the development of distributed computing frameworks, such as Apache Hadoop and Apache Spark, which allow for the processing of large datasets across a cluster of machines, allowing for parallel processing and fault tolerance, making it possible to handle big data even if a single machine fails on the cluster. Apache Hadoop, which was one of the first distributed computing frameworks, is comprised of the Hadoop Distributed File System (HDFS) for storing data across multiple machines, MapReduce for processing data in parallel, and YARN for resource management. However, Apache Spark has emerged as a more efficient and faster alternative to Hadoop, due to its ability to perform in-memory computation, which drastically reduces the time taken to process data compared to Hadoop's disk-based approach [2]. This makes Spark more effective as it avoids repeated disk I/O cycles, allowing for faster processing for iterative or multi-stage computations.

2.2 Aims and Goals of the project

This project aims to use a large text-based dataset, specifically a 100 GB file comprised of Wikipedia articles, to implement two programs using PySpark, the Python API for Apache Spark. The first program is a word count application that counts the number of times each word within each page of the dataset appears. This is quite a standard benchmark to examine distributed processing behaviour and performance. The second program performs distributed grep, which searches for a specified word or pattern within the Wikipedia articles and returns the titles and all occurrences of that word or pattern. Both programs require the data to be cleaned and pre-processed before the main functionality can be implemented. Some of the challenges that arose during the implementation of dealing with such a large and complex dataset include cleaning and normalising the data, data partitioning and scalability across the cluster nodes.

A huge part of the project is the university cluster, where the Spark jobs were run across multiple nodes, allowing for the Wikipedia dataset to be processed in parallel across the cluster. This allowed for an evaluation of the performance and scalability of the programs by tuning the Spark-submit parameters, such as executor memory, number of cores and partitions of data. These experiments provided valuable insights into the performance of Apache spark for large scale text processing tasks, showing how a different number of resources and varying dataset sizes can affect the runtime of the implementation.

2.3 Extended Scope and Contributions

In the initial phase of the project two programs were implemented using Apache Spark over a large Wikipedia dump and evaluated. These programs were a distributed word count and a distributed grep workload. They were chosen in order to expose the core distributed system costs and therefore acted as an effective baseline for the performance characterisation of Spark.

The later phase in the project looked to move beyond text analysis and delve into the world of statistical analysis in the form of structured numerical analytics and unsupervised learning over the NOAA weather dataset. This extension was included to demonstrate that Spark can support multiple analytical paradigms. The resulting report therefore covers the behaviour of shuffle-heavy text workloads as well as the behaviour of aggregation and machine learning driven workloads under a range of controlled cluster configurations.

The overall contribution is a technical evaluation of Spark as a general-purpose cluster analytics engine, with a range of implementations throughout the course of the project from data cleaning to clustering with the end goal to show the usability and effectiveness of Apache Spark in massive scale data analytics.

2.4 Planning and time-scale

This is the timeline I followed through the course of the initial phase of the project:

- **Week 1:** Initial project setup phase began and started to learn Apache Spark via the PySpark library. Reviewed some core Spark concepts and identified the Wikipedia XML dump as the main dataset to use.
- **Week 2:** No access to cluster so began local prototyping using a small CSV dataset to confirm PySpark installed and could practise using DataFrame operations.
- **Week 3 - 4:** Implemented the Wikipedia XML reading via a specialised package. Then filtered relevant pages to text based operations and ran a cleaning regex pipeline. Validated the clean outputs using small Wikipedia subsets.
- **Week 5 - 6:** Implemented the Word Count program using the DataFrame API. Verified correctness on a 1 GB subset, the proceeded to up-scale and use the larger datasets I stored in HDFS once given access to the Big Data Cluster.
- **Week 7 - 8:** The Distributed Grep program was implemented using the same cleaning pipeline, then applying a word matching function over the cleaned text data. Implemented this over the larger dataset size.
- **Week 9 - 10:** Designed and executed a structured performance analysis plan for the programs on the Big Data cluster. Kept a track of the runtimes of multiple different configurations used with `time spark-submit` and a variety of dataset sizes.
- **Week 11:** Data gathered on the excel sheet translated into plots using Matplotlib and pandas library.

2.5 Later-Phase Timeline and planning

- **Week 12 - 13:** Begin the next phase of the project by selecting a suitable large structured numerical dataset for the statistical analysis program and then go on to research an appropriate PySpark testing approach. Start implementing the core statistics functions and then validate the logic using local tests.

- **Week 14 - 15:** Develop the percentile-based bucket and frequency distribution stage of the program, focusing on a method to avoid repeatedly scanning the dataset. Then bring the separate statistical analysis functions together into a single main program that reads the NOAA weather dataset, to then perform the large scale analysis. After that the goal was to output the results, and generates a frequency distribution graph as part of the visualisation requirement.

- **Week 16 - 17:** Run the statistical analysis program on the larger weather dataset and confirm that the full distributed workflow executes successfully. Then move on to planning and beginning the final data mining stage of the project, focusing on implementing the K-Means clustering algorithm and deciding which weather-based features would produce meaningful cluster groupings.

- **Week 18 - 19:** Continue the K-Means implementation by generating the first clustering outputs and visual representations, then review whether the resulting groups are sensible. Refine the feature representation where necessary so that the final clustering output produces clearer and more meaningful patterns.

- **Week 20 - 21:** Improve the handling of output files from HDFS so that the results can be managed more easily for later local visualisation. Establish a clearer workflow between the Big Data cluster computation and the map visualisation and scatter plot presentation stage.

- **Week 22 - 23:** Begin and continue a structured performance testing stage for the statistical analysis program by creating a repeatable baseline process, then testing different executor configurations and partition counts. Compare the runtimes across these configurations to identify the most effective setup for the workload. Present the collected results in plots to be further analysed.

3 Background Theory

3.1 Big Data Characteristics and Challenges

This project's key goal is understanding how to harness and process the knowledge that is hidden within the large, complex collections of information. Complicated and massive datasets that traditional data management systems cannot handle are referred to as big data [4]. A 100 GB XML Wikipedia dataset is used for the early deliverables of this project, which is perfect to showcase the power of distributed processing using a combination of Apache Spark and the university's cluster. Clearly, this massive dataset would prove difficult to handle without a distributed approach.

Some challenges that come with dealing with massive datasets can be cleaning the information. This can be broken into segments that can become very involved due to the quantity of data being processed. Data cleaning systems must be put in place due to the rising importance of having quality data [5] to perform experiments to analyse what the collection of data is indicating. Unclean data can be very costly due to the amount of processing power, resources and time that would need to be invested to end up with valuable data that can be used for further analysis [5].

A few other difficulties are the processing throughput and system bottlenecks, as the size of the massive set of data alone will not be a feat considered impossible as the size of data being processed can be stored on a modern machine. The more important issue is how long it takes to repeatedly read, parse, transformations and process that data via the cluster. This would be when disk I/O can become a bottleneck, as reading datasets at a large scale from disk takes time and repeated scans, parsing and writing intermediate results all a processes that slow down the time to completion. Network transfer can further limit performance as stages within the computation can be particularly expensive, which could come in the form of shuffle-heavy stages.

3.2 Distributed Computing Models

3.2.1 MapReduce

Map and Reduce are key concepts to understand when delving into how distributed computation functions. These concepts were popularised by an open-source implementation of the MapReduce programming paradigm, called Apache Hadoop. The Map function has the role of taking raw input data, then processes it to generate a set of intermediate key-value pairs [6]. The Reduce function then takes the output of the Map function and merges the values that share the same key [6]. This step aggregates the data to produce a final smaller output, and when this is put into effect over large tasks, they can be broken down into independent sub-tasks that run in parallel significantly reducing the time taken to process the dataset.

Although MapReduce was a great and effective way to process massive datasets, it became glaringly obvious to researches at UC Berkeley that MapReduce was becoming inefficient for iterative computing jobs [2]. This bought about the idea to create Spark in the aims to make a simpler and faster implementation [2].

Apache Spark builds upon the foundational concepts of distributed, parallel processing across a cluster of machines, and as seen in the project, runs over the Hadoop Distributed File System (HDFS). MapReduce writes intermediate results to disk after each Map and Reduce stage, which is quite slow due to disk I/O latency. Spark performs in-memory

storage for intermediate computations that allow for a much faster rate of processing large quantities of data [2]. A basic concept within Spark called Directed Acyclic Graph (DAG) is an engine that helps cyclic data flow [7]. By using a DAG scheduler that builds a complete lineage of transformations before execution, this allows Spark to combine multiple operations into a single stage and have fewer shuffle operations [7]. Lastly, fault-tolerance is another key feature that Spark achieves via Resilient Distributed Datasets (RDDs), which track the sequence of operations so if a partition is lost Spark can easily recompute only the missing data using that sequence to avoid unnecessary instructions. Whereas, MapReduce ensure fault tolerance by writing intermediate data to disk and using pings to notify the master node of whether or not the computation has succeeded [6], which is quite sluggish in comparison to the speed that Spark provides.

3.2.2 HDFS and Cluster Architecture

Hadoop distributed file system (HDFS) is a "platform for storing, managing and distributing data" [9] across the machines within a cluster. The main objective that HDFS has is to store those large quantities of data and to supply a high throughput of data within a system [9]. Typically, the partitions are split into block sizes of 128 MB which can be distributed in parallel to allow for parallel processing through the various DataNodes in the cluster. Another key feature possessed by HDFS is the replication storage model, which allows for the blocks of data to be replicated three times and stored in three different DataNodes [9], which provides security for fault tolerance if a DataNode was to fail or be taken out of the cluster for maintenance.

A cluster is made up of a multitude of nodes that work in parallel to complete computations. A master node, also known as the NameNode in HDFS, can be seen as the 'brain' of the cluster, with its primary responsibilities being job management, scheduling tasks and coordinating the activities of all worker nodes [6][8]. The worker/slave nodes, known as DataNodes in HDFS, contribute to the task by storing data via a distributed file system such as HDFS and executing the computations assigned by the master node [8], they then report back to the master node, which can aggregate the final results and inform the user that the job they submitted is has been completed.

Although HDFS is the storage system used within this project, Apache Spark is not bound to HDFS alone. In cloud development, object stores like the Amazon S3 architecture is a common alternative to local storage systems [2], which gives Spark the applicability to be utilised across a wide array of different development environments. This is another key feature in Spark's design that allows it to be as scalable and pragmatic to use while processing massive scales of data.

3.3 Apache Spark Overview

3.3.1 Spark Architecture

Spark has many components that combine to process large datasets with an incredible level of efficiency. This figure shows the main mechanisms that activate during the execution of a Spark application, including the driver node, YARN (a cluster manager), worker nodes and the distributed storage layer from which partitions of data are read.

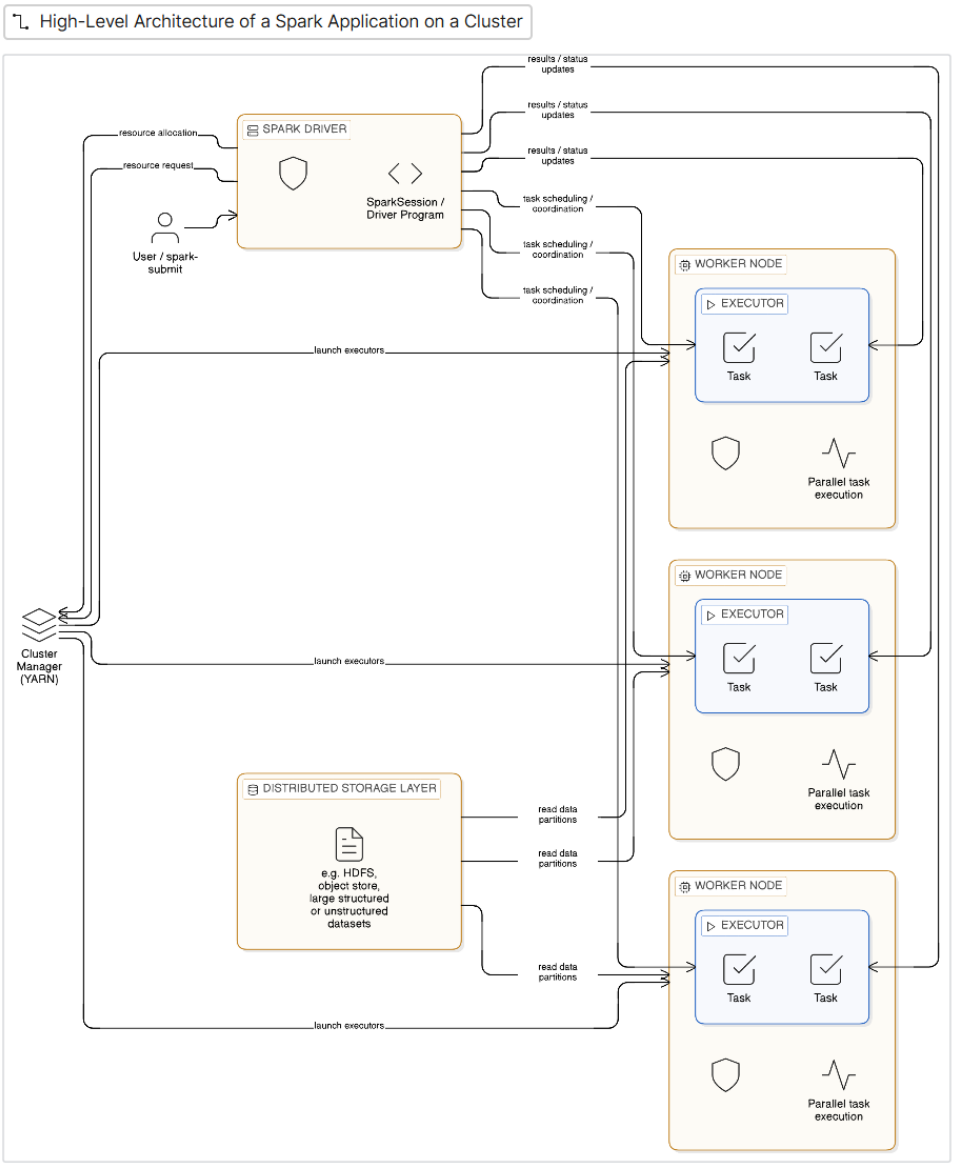


Figure 1: High-level architecture of a Spark application on a cluster.

Sparks operational architecture is similar in physical layout to Hadoop, in that it uses master-worker nodes but its highly optimised and more effective for iterative jobs. Spark's superiority for iterative jobs stems from its ability to retain data in-memory, and since many machine learning algorithms use a function many times on the same dataset [10], this feature has revolutionised data processing which is seen by Spark outperforming Hadoop by 10x [10]. To do the same job, Hadoop would need to read and write from disk multiple times, which significantly slows down the processing time [10].

At the core of Spark's great performance improvements are several different components like the driver, executors and the cluster manager. The driver is the central brain of a Spark application as it runs the main function and is responsible for maintaining the information about the Spark application. Primarily, the driver negotiates resources from the cluster manager and work scheduling tasks on the executors [2]. The Directed Acyclic Graph (DAG) scheduler is built and optimised by the driver, separating the dependencies between the various stages of the job and creating a roadmap for execution [11][2] by modelling the data flow and intelligently groups multiple transformations into

a single processing unit called a stage. A stage is a set of tasks that are executed in parallel across the cluster [2].

The cluster manager is a vital component that is responsible for acquiring the computing power across the cluster and allocating them to various Spark jobs [2]. Spark currently supports four cluster mangers, which are the internal stand-alone cluster manager, Apache Hadoop YARN, Apache Mesos and Kubernetes [2]. Finally, executors within Spark are run on each worker node and are tasked with processing the data and returning the results to the driver.

3.3.2 RDDs, DataFrames, and Datasets

The key abstraction within Spark is the Resilient Distributed Dataset (RDD), a foundational data structure which is a read-only collection of objects that are partitioned across the nodes of the cluster and can be used in parallel [10]. One of the benefits provided by RDDs include their immutable nature, which allows for systems to alleviate the burden of slow or faulty nodes within the cluster, as they can simply recompute lost partitions using the lineage of transformations [12]. This process would then be carried out by a healthy worker node, ensuring that the overall computation can continue without any significant delays.

DataFrames and Datasets are higher-level concepts built upon RDDs that provide a more user-friendly interface for working with structured data. While RDDs are quite flexible and can handle unstructured data [12], they lack inherent structure forcing Spark to treat them as generic objects. DataFrames are similar to tables in a relational database, using rows and columns, allowing for a more intuitive way to work with data's content. Datasets are a type-safe version of DataFrames, providing "compile-time type safety" [2] which helps catch errors during development.

3.3.3 Transformations, Actions, and Lazy Evaluation

The Spark execution model is primarily based on two operations: transformations and actions. Transformations are operations that reconstruct a Spark DataFrame into a new DataFrame without effecting the original data [2], which gives it the property of permanence in the new structure. For example, an normal operation such as select() on a DataFrame will not change the original DataFrame [2], but will display the new results of the operation as another DataFrame. A key feature that transformations possess is that they are lazy, meaning when you write a transformation Spark would not execute the operation immediately; it merely creates the record of steps in the Directed Acyclic Graph (DAG) and does not trigger any computation on the data.

There are two types of transformations that can occur: narrow and wide [2]. These are crucial to Spark's ability to efficiently parallelise the workload. Narrow transformations are operations where each input partition maps to exactly one output partition [2]. These transformations are highly efficient due to their ability to execute without requiring data movement across the cluster and can be performed on the worker nodes where the data resides. On the other hand, wide transformations are where a single input partition contributes to multiple output partitions [2], which requires Spark to move and exchange data across the network between different worker nodes, known as a Shuffle.

This is where actions come into play, actions are operations that instruct Spark to start the actual computation on the dataset from the recorded transformations [2]. The result of these computations can then be returned to the driver node or be directly parsed

to a storage system like Hadoop's Distributed File System (HDFS). Through a function like show() being executed within the code, the plan laid out by the DAG would then be executed over the dataset.

As we know all transformations throughout Spark are evaluated lazily, meaning that the results of these computations are not processed instantly and that they are instead recorded as a lineage [2]. Lazy evaluation is the key mechanism within Spark that waits for an action to occur before performing any computation. The benefit of this approach stems from the Catalyst Optimiser being able to perform global optimisation across the entire pipeline [8], which then allows Spark to look at the complete DAG of operations and determine the most efficient path to take. This can be done by Spark joining or pipelining operations within the code and assigning them to a specific stage.

3.4 Performance Concepts

3.4.1 Parallelism, Partitions, Shuffles

Parallelism is a key factor as to why Spark is so efficient when it comes to processing massive quantities of data. It is achieved by spreading the workload across a number of machines, executing multiple tasks simultaneously instead of having a single machine carry the workload by itself [14]. The result of this dispersed approach to processing the large dataset is a significant reduction in the time it takes to complete the job, as multiple nodes work together to handle different parts of the data concurrently [14].

Some core laws heavily related to the experiments I conduct are, Amdahl's law, which states that the speed-up achieved by a parallel system is limited by the time required for the portion of the workload that is sequential [15]. This suggests that the benefits of parallelisation are not consistently scaling and will eventually flatten out. The formula presented for Amdahl's law [16] is:

$$S(n) = \frac{n}{1 + (n-1)f}$$

This formula shows the rate of increase in performance, S(n), as the number of processors, n, is increased with f being the part of the job that is not parallelised. The main implication of this law is that it explains the development of diminishing returns as there will be a fixed amount of time taken by the sequential shuffle operations within the Spark job, resulting in the limitation of the job's runtime.

Another important law is Gustafson's law, which describes how performance can be improved when the size of a problem is scaled with the number of processors that are added to the workload [16]. This law is relevant as it suggests that with more resources available within the cluster, larger datasets can be processed more efficiently, whereas Amdahl's law assumes a fixed workload is used. Additionally, this is a key aspect of why a distributed framework like Apache Spark is so well suited for analysis of big datasets. However, Gustafson's law does not simply mean that every workload will scale efficiently simply because more data is added. It is highly dependent on whether the work continues to remain parallelisable and is not overwhelmed by additional shuffle costs or communication overhead. The formula for Gustafson's law [16] is:

$$S(n) = n - \alpha(n-1)$$

This formula indicates that the speed-up, S(n), increases linearly with the number of processors, n, minus the fraction of the workload that inherently cannot be parallelised, represented by α. It emphasises that as more processors are added, the overall performance can improve significantly, provided that the non-parallelisable part remains minimal.

To effectively process the data in parallel, Spark must first divide the data into smaller logical chunks called partitions. Each of these partitions can then be processed independently and in parallel across the slave nodes within the cluster [2]. The number of partitions directly influences the level of parallelism that is possible, as more partitions allow for more tasks to be executed simultaneously. However, too many partitions can lead to a large overhead in managing these tasks [2], as each task also comes with its own setup and finishing operations.

As mentioned earlier in the report, a shuffle is a costly operation in which data exchange is required by wide transformations such as `groupBy()`, `join()` and `sort()`. This forces data to be moved across the network to be aggregated, which creates a barrier-like stage that requires synchronisation because later tasks must wait for shuffle outputs to be materialised, in the otherwise parallel processing that Spark provides. This is due to the fact that all tasks must complete their shuffle operations before the next stage can begin [2]. As a result, they can dominate the cost of large-scale aggregation, that has an effect on the runtime performance. This then makes it crucial to minimise the number of shuffles involved in a Spark job where possible to decrease the performance overhead.

3.4.2 Memory Management

Efficient memory management is a critical aspect of Spark's performance, as it directly affects the way in which data is processed and stored during the computation. Spark uses a unified memory management model that allocates memory for two main purposes: execution and storage [2]. Execution memory is a space reserved for the main computation, such as shuffles, joins and sorts. Storage memory is dedicated to caching and persistent storing of the partitions originating from the DataFrame to be kept within RAM.

3.4.3 Distributed Statistics and Approximation

There are several descriptive statistics that can be computed efficiently computed over a distributed system, for example a count, a sum variable, the minimum and the maximum. However, exact quantiles such as the median become much harder to calculate efficiently at a larger scale because they would require either a global ordering within the dataset or an equivalent global coordination, which in Spark would normally increase the number of shuffles and lead to additional memory pressure.

Spark therefore provides approximation tools for quantiles. At the DataFrame level `approxQuantile` exposes quantile estimation with a user determined error parameter [18], where setting that argument to zero requests the exact quantiles. This would be extremely expensive for large datasets. For SQL style aggregation, Apache Spark provides an alternative via the function `percentile_approx(col, p, accuracy)` [2], which has the trade of memory usage for accuracy using an accuracy parameter and avoids the full cost of an exact global sorting to achieve an absolute outcome. These choices show a common big data principle, when the dataset is large enough, a bounded approximation can be a better engineering trade-off than having to force a costly global data movement to gain an exact statistic.

This form of approximation avoids having to store and sort the entire large dataset [17]. Instead, allowing an answer to be achieved with a margin of error, making it suitable at large-scales and in distributed analytics.

3.4.4 Spark Submit Parameters

The spark-submit command is used in order to run the Spark application over the cluster, There are a variety of adjustable parameters used within this command to define the computational resources being allocated to the job. This will be the fundamental to be able to finding performance testing aspects to the programs within this project. Some of the resource allocation parameters include the number of executors, which dictates the number of executors to launch on the cluster's worker nodes. There is also the executor cores parameter, which specifies the number of CPU cores to be assigned to each executor. This is another factor that that affect the level of parallelism as increasing this parameter allows for tasks to run concurrently on an individual executor. Another component would be the executor memory, which assigns the amount of RAM allocated to each executor.

3.5 Columnar Storage and Analytical Query Performance

Traditional row-oriented storage layouts write each record as a continuous sequence of fields, which is more suited towards traditional workloads that frequently read or update entire rows. On the other hand, for cases where analytical workloads are required, they usually tend to operate on a smaller subset of the columns across a large array of information that may not be entirely relevant to the task at hand. This could lead to filtering by one field and aggregating another. In a row-based layout, the system must still read every field in every row in order to obtain information that may only be stored in one or two relevant columns during the course of the query. This would result in significant amounts of unnecessary I/0 transfers [24].

Column-oriented storage addresses the issue at hand by storing each column independently, this would mean the system can read only the columns needed for that particular operation. This drastically reduces the volume of data transferred from the disk, which is significantly beneficial for a set of large-scale analytical queries where I/O is usually the main bottleneck [24]. Furthermore, due to values within a column sharing the same data type, columnar formats can apply more effective compression techniques like run-length encoding and dictionary encoding, which further reduces the amount of data that needs to be read and processed during an intense workload run [24].

Parquet is a file type that is widely used due to its columnar file format and can be utilised within the Hadoop and Spark ecosystems. It was inspired by the record-shredding and assembly approach seen in Google's Dremel system [25]. Parquet stores data in a columnar layout with metadata embedded within, which includes the schema and basic column statistics. These statistics allow Spark to skip entire blocks of data which do not align with a filter condition for that query, this technique is known as predicate pushdown. Combined with column pruning, this means that Spark reads only the columns referenced in a query and this further proves how operations on a massive dataset can nullify reading portions of a large chunk of data stored.

Within Apache Spark, the Catalyst Optimiser harnesses these columnar properties when constructing the physical execution plan [13]. When the input data is stored in a row-oriented format like a CSV, Spark has the ability to internally represent DataFrames

in a columnar memory layout through the Tungsten execution engine, allowing for efficient vectorised processing to take place. Although, the full extent of the benefits of predicate pushdown and column pruning are realised when the underlying storage format is a Parquet, as it is a storage format that has the columnar property natively.

3.6 Bucketisation for Scalable Frequency Distributions

The traditional method to categories a variable with a frequency distribution can be implemented by iterating over specified intervals and repeatedly filtering and counting how many variables fall into each group. In Spark, this method of recurring filtering runs the risk of triggering a multitude actions resulting in repeated scans being enacted over the large dataset, becoming very inefficient with large workloads.

Spark MLlib provides the `Bucketizer` transformer [19] as a way to assign each input into an interval based group defined by a sequence of split points. Bucket assignment is executed over the cluster as a distributed transformation, which leads to a standard aggregation resulting in the frequency distribution within a single pipeline. This approach lines up well with Spark's execution system: one pass to assign the buckets then another aggregation to count in order to avoid the expensive and frequent scans.

Conceptually, bucketisation is also a feature-engineering step that is used to turn numeric values that can take many possible values into a small number of ranges of bins, which would serve as an input representation for modelling via plots.

3.7 Machine Learning and Clustering

3.7.1 Unsupervised Learning and K-Means

K-Means clustering partitions observations into k groups by iteratively assigning points to their closest centroid and recomputes where each centroid should be. This process is repeated until the centroids stop moving much, meaning that the goal to make points inside each cluster as similar as possible has been achieved.

Standard K-Means++ is the algorithm that picks desirable starting centroids in order to improve the resulting outcome. Although, this is a desirable property, it does so sequentially. Over a large dataset, sequential steps are to be avoided as and when possible. Apache Spark's implementation of the K-Means data mining algorithm is far more scalable as the design is specifically for large-scale distributed systems to reduce the bottleneck caused by the sequential initialisation [21].

In Spark's DataFrame-based ML API, `KMeans` is an estimator that produces a fitted KMeansModel [20][22]. Key hyper parameters include the number of clusters generated, k, then the maximum iterations it would take to find the final centroid position, as well as the convergence tolerance and finally the distance measure. The resulting training objective enables direct comparison between the varying k values and features representations generated as a result of the algorithm.

Due to K-Means being sensitive to feature scaling as well as feature choice, the interpretation that clusters have is function not only of the algorithm but of the engineered feature representation. This motivates an iterative workflow where the clustering outputs are visually checked and the feature set refined in the case that the clusters appear to be dominated by one variable and cloud out any other variables added, making them irrelevant or lack a definitive structure when represented visually.

3.7.2 Clustering Evaluation Metrics

In addition to a qualitative inspection via the maps and scatter plots generates, the clustering quality can be reported using internal validation indicators. Spark provides `ClusteringEvaluator` [23] with a Silhouette score option, where values closer to 1 indicate tighter clusters that are more separated from other clusters on the same graph. The K-Means training cost, which would be the sum of the squared distances to the closest centroid, is also exposed within Spark and can be used to support the elbow-style selection of k.

These metrics are also useful in this project as they allow the impact of feature selection and hyper parameter change to be quantified along with the analysis that is presented with the visual interpretations.

4 Implementation and Experimental Work

4.1 Architecture of the Spark Application

The two programs implemented in this project share a common pipeline structure due to their nature. Some of those aspects are the initiation of the Spark environment, reading the raw XML dataset, filtering and cleaning which is followed by the specific logical operations based on what the program was implemented to achieve. This structure seems quite a handful but is actually made very efficient due to Spark's Lazy evaluation, which ensures that all these tasks are pipelined into an optimised execution plan.

The physical architecture involves the Big Data cluster used through the course of the project, it consists of 4 main components: the HDFS storage system, the Spark driver, the YARN cluster manager and the worker nodes within the cluster. HDFS is used to store the massive Wikipedia dataset on a distributed system. The Spark driver is the component that runs the python script, building the execution plan and coordinates the Spark job. YARN is the cluster manager, which works behind the scenes to allocate the resources requested in the prompt. Finally, the worker nodes run the parallel tasks, performing the computations and returning the results to the Driver node for the final results.

The project primarily relies on the DataFrame API within PySpark, which is a high-level abstraction chosen for its benefit presented by Spark's Catalyst Optimiser. It constructs a highly efficient execution plan and due to the tungsten engine providing faster execution rather than manual RDD code.

4.2 Interim Data Source

The main source of data used to test both applications was the Wikipedia XML dump (around 100 GB), which was sourced via wget into the Big Data cluster, uncompressed and finally stored within the Hadoop Distributed File System (HDFS). Experimental runs were conducted with subsets of this 100 GB dataset which range from 10 GB up to 90 GB to analyse performance scaling due to its significance in the Big Data space.

Since the data source I chose is XML, I found a package that I utilised within the Spark execution which allows the files to be read as Spark DataFrames. Therefore, the `com.databricks:spark-xml` library was used. Some key options also included were the rowTag option to be implement over each `<page>` tag to correctly segment each Wikipedia article.

4.3 Final Data Source

Following the interim use of a Wikipedia Dump, the final dataset adopted for the later stage of the project was the NOAA Global Historical Daily Dataset (1900 - 2025) which was also around 100 GB in size. This represented a more suitable final dataset for the statistical analysis and data mining, which involved clustering presented in problem 3 and 4 of the project guidelines. This is due to the dataset having structured numerical data rather than unstructured text, which would not be as useful for this stage of the project.

The weather data is an integrated archive of daily land-surface climate summaries compiled from various different contributing sources and then subjected to a series of quality assurance procedures. In addition to the observations themselves, NOAA provides

accompanying station metadata and data-quality information. This is important when assessing the reliability, consistency and understanding that can be gauged from the records used later in the computation. Furthermore, this level of assurance provided with the quality of the data allowed me to assess that the output would be meaningful for analytics.

From an implementation perspective, the final weather dataset aligned well with the project requirements as well as Spark's structured processing model. The records were taken into Spark DataFrames using a specified schema, which enabled column-based transformations, filtering and aggregation with the final stage being an output to a CSV file for later use when representing the results visually.

4.4 Pre-processing and Cleaning Pipeline

The pre-processing stage was identical for both programs, designed to normalise text and remove as much Wikipedia-specific markup and noise, ensuring the most accurate tokenisation and pattern matching with each respective program. Some steps taken in order to gather the correct information within the large Wikipedia XML dataset were to initially filter the pages by removing anything that was non-article pages, the to remove any redirect links as well as to get rid of empty pages.

After this selection and filtration process was complete, the remaining data, consisting of pages of Wikipedia articles, could now be used to start the cleaning process in order to make them useful for the programs being built. The cleaning process began with a case normalisation along with a multitude of regex alteration functions on the text data, with the intent to remove things like Wiki links, templates and HTML tags present in the uncleaned text field.

All the cleaning performed was accomplished using PySpark's SQL functions (such as `regexp_replace()`, `lower()`, etc.) over using User-Defined Functions (UDFs) as they provide better performance and optimisation over the PySpark application. A important reason why this occurs is due to Spark SQL functions being understood by the Catalyst Optimizer, responsible for those added benefits of performance.

4.5 Word Count Implementation (Problem 1)

The Word Count program performs a large-scale statistical analysis over the Wikipedia Dump to determine the frequency of every word across the entire dataset. These are the steps taken to create the Distributed Word Count:

1. **Load the XML data from HDFS:** the Wikipedia dump is read from the HDFS file path using the park XML data source treating each `<page>` element as a record.

2. **Filter valid articles and select:** filtering and selecting the title and text columns within the pages.

3. **Cleaning the text content by:**

    - Converting all text to lowercase
    - Removing wiki-markup, HTML tags and URLs using regex
    - Filter out any non-alphabet characters

4. **Tokenise the cleaned text:** the cleaned text is then split into an array of words.

5. **Explode each word into rows:** the array is transformed so that each word has its own row, allowing for distributed aggregation.

6. **Filter out any invalid or unnecessary words:** removing any words that are would not be necessary in the final word count.

7. **Aggregate counts per word:** each word is grouped and counted to compute a global frequency for each unique word. This process is a wide transformation and triggers the shuffle phase.

8. **Ranking the results:** the counts are sorted into descending order and the top 20 results are taken to be displayed in the output (as a way to verify the computation was completed).

This is a pipeline and flow of the program as the code runs on each executor, in reality this code is run multiple times during the Spark job in order to clean the large dataset as the partitions are cleaned and their results are then aggregated within the driver node.

4.6 Distributed Grep Implementation (Problem 2)

The Distributed Grep is a far more efficient program as it avoids the global shuffle required to count the words in the Distributed Word Count program. The program aims to find a keyword or patterns within the Wikipedia dump.

1. **Define the keyword:** the keyword is defined to later be used within the matching logic to extract each occurrence.

2. **Repeat steps 1 - 3 from the Word Count program:** the problem requires for the text data to be utilised in order to find the occurrences of the word within the large dataset, the loading of the dataset and cleaning processes on each page from the first program can be implemented again.

3. **Filter matching documents:** the dataset is then filtered so only the rows (article pages) with the matching word is kept.

4. **Select output fields:** the resulting output is minimised down to the word itself and the title page in which it was found.

Unlike the Word Count program, the Distributed Grep program is primarily a filtering workload; it does not have the same need to perform a large number of shuffle operations as there is no global aggregation that takes place. Cleaning the data remains consistent throughout the process as its essential to perform these implementations like Word Count and Distributed Grep on a set of data without any of the usual markup and metadata within Wikipedia pages as it can provide false positives during each of these processes.

4.7 Experimental Design and Execution

4.7.1 Small-scale Tests

The main reasoning behind starting with miniaturised datasets (that were also Wikipedia articles) was to ensure the pipeline generalised to the full dataset before being given access to the Big Data cluster on the universities servers. The debugging of code before performing any sort of performance analysis was crucial as it would be a lot more inefficient to conduct on the larger datasets.

Initially, these smaller tests were performed locally as a way to learn what spark was capable of. Then, they become useful to clean the data on the cluster as a way to validate that the cleaned data was suitable for either the Word Count or Distributed Grep operation. This reduced any risk of spending cluster time on the large-scale runs, which would take quite long to process the data of that scale.

4.7.2 10 GB to 90 GB Tests

This phase of the experiment was designed to establish how runtime changes as the dataset size increases while controlling the parameters of the cluster in order to evaluate the results. The independent variable with every configuration on the cluster was the dataset size (10 GB, 20 GB, up to 100 GB), while all other factors were kept fixed:

- **Code Consistency:** the same code, which included the cleaning pipeline and program logic were utilised across each subset and for the distributed grep in particular the key word "queen" was kept consistent.
- **Resource Configuration:** for a given experimental configuration the parameters being allocated across the cluster (executors, cores and memory) remained constant to isolate the impact that data volume had on performance.

These subsets were generated from the original Wikipedia dump (100 GB XML dataset) and stored as a individual paths in HDFS. Each of the run times were repeated to ensure the initial result was reliable and consistent.

4.7.3 Full 100 GB Tests

The final-scale execution protocol focused on running the end-to-end pipeline on the full 100 GB XML dataset stored in HDFS. This stage served as the ultimate test of the distributed cluster's along with the implementations created.

Jobs were submitted through the YARN cluster manager using a standardised `spark-submit` template. To ensure the operational stability during execution, long-running 100 GB jobs were executed within tmux sessions, and recorded the outputs produced after completion.

Monitoring was performed via the Spark UI generated during execution, specifically focusing on:

- **The Jobs Page:** to observe stage progression and the impact of the shuffle phase in the Word Count job in particular.
- **The Executor Page:** to monitor task distribution and memory usage across the worker nodes on the cluster.
- **The DAG Visualisation:** to confirm that the Catalyst Optimiser correctly pipelines the multiple regex transformations into an efficient stage.

4.8 Statistical Analysis Implementation (Problem 3)

The statistical analysis program was designed to compute a few statistics as well as generate an output for a frequency distribution over the NOAA Global Weather daily dataset. Unlike the text heavy Wikipedia workloads used in the first two problems, this program operates on structured numerical data. This would require a different approach to cleaning and processing the data.

The initial step was to define an explicit schema for the NOAA dataset using PySpark's `StructType` and `StructField` constructs. This schema contains the column names and data types for each field in the CSV files, including the station identifier, date, element type and the recorded data value. The rational behind defining the schema upfront rather than letting Spark infer it allowed for two objectives to be fulfilled including: avoiding an additional pass over the large dataset that inference would require as well as the fact it enabled Spark's Catalyst Optimiser to apply the column pruning and predicate pushdown concepts more efficiently as discussed within the columnar storage theory [24][13].

These are the steps that were taken to create the Statistical Analysis program:

1. **Define the NOAA schema:** an explicit schema was defined using `StructType` to specify the column names and data types, as a way to remove the additional cost of schema inference over the large dataset.

2. **Load the CSV data from HDFS:** the weather dataset was read from HDFS using the schema, which allowed Spark to apply column pruning.

3. **Filter and clean the data:**

   - Retain only rows where the element type is 'TMAX'
   - Convert the raw values obtained from tenths of a degree to a standard Celsius by dividing the number by 10
   - Drop any null temperature values
   - Enforce a sanity bound excluding temperatures outside negative 100 to 60 degrees C

4. **Compute summary statistics:** Once all the data is present the count, minimum, maximum and median were calculated in a single distributed aggregation. The median was estimated using `percentile_approx` to avoid a costly global sort across the cluster [18].

5. **Generate frequency distribution:** The `Bucketizer` transformer assigned each temperature value into one of ten equally spaced intervals in a single distributed action, which is then followed by a `groupBy` aggregation to count up each bucket [19].

6. **Attach range labels:** temperature range labels were created and joined to the bucket identifiers to produce a output for visualisation.

7. **Export the output results:** the summary statistics and frequency distribution were written out as CSV files, which were easier to inspect and transfer to the local visualisation stage.

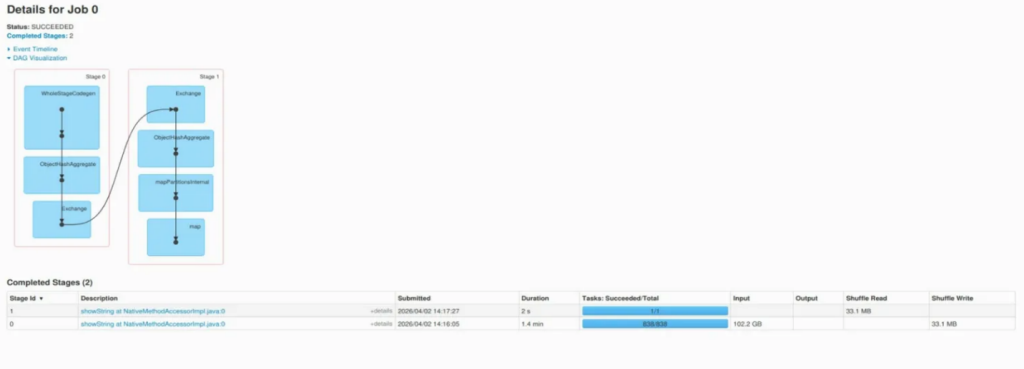


Figure 2: Spark UI DAG visualisation for the statistical analysis summary statistics job.

The DAG visualisation captured from the Spark UI during an execution of the statistical analysis program confirms that the Catalyst Optimiser pipelined the filtering, unit conversion and null removal into a single narrow stage, executed across 838 parallel tasks reading the input data. The stage boundary appears at the Exchange node, which represents the shuffle required by the aggregation to compute the summary statistics within the program. Stage 1 then completes the grouping in a single task. This shows the efficiency of the pipeline as the bulk of the work occurs in the parallelised narrow stage with the shuffle overhead being kept minimal relative to the input size.

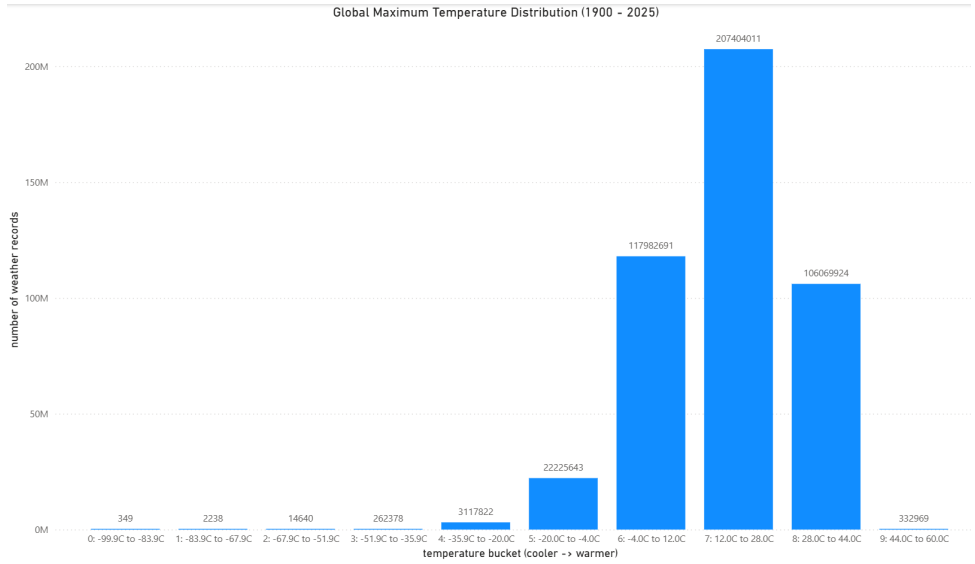


Figure 3: Frequency distribution of maximum temperatures across the NOAA dataset.

This pipeline differs from the Wikipedia programs as the cleaning stage relies on a value based filtering and numeric conversion rather than with the first two programs where regex-based text normalisation was utilised. However, the underlying principle stays the same as the transformations are recorded lazily and only executed when the action like writing the outputs are trigged during the computation across the cluster.

4.9 K-Means Clustering Implementation (Problem 4)

The last program utilised a K-Means clustering algorithm over the NOAA weather data to group weather stations into k number of temperature/climate zones based on their

temperature behaviour. This represented the data mining component of the project requirement and showed the use of Spark MLlib for unsupervised learning at a large scale.

This implementation in particular encapsulated within a `ClimateClusterer` class that was able to manage the Spark session, the schema definition, the data cleaning component, the feature extraction as well as the clustering stages. This modular design approach allowed the same class to be reused across the local testing environment and the Big Data cluster execution once those tests had completed successfully, with the only difference being whether a Spark session was created locally on a machine or passed in with a spark-submit command to configure the clusters resources to the desired metrics.

These are the stages taken to create the K-Means Clustering program:

1. **Load and clean the weather data:** the same NOAA schema and TMAX filtering logic from the statistical analysis program was reiterated to load and clean up the dataset.

2. **Extract features per station:** the cleaned temperature records were aggregated per station to compute the four following features:

   - Average temperature
   - Maximum temperature
   - Minimum temperature
   - Temperature range

   Any stations with missing values were dropped at this stage to avoid voiding results.

3. **Assemble feature vector:** the four temperature features were combined into a single mathematical vector using `VectorAssembler`.

4. **Scale the features:** `StandardScaler` was applied in order to normalise the feature values so that no single feature is able to dominate the distance calculations due to difference in the scale [20].

5. **Train the K-Means model:** the model was trained with k set to 5 and a fixed random seed for reproducibility. The algorithm iteratively assigned stations to the nearest centroid and recomputed them until convergence was achieved [21].

6. **Assign cluster labels:** each station was assigned a cluster identifier by the trained model. The final output retained the station name, the temperature features and the attributed temperature features and the assigned cluster.

7. **Export the clustered output:** the results were written to HDFS as a CSV file and later transferred in order to visualise locally.

An earlier iteration of the clustering included a precipitation weather feature. Although, after reviewing the visual output on the map, it became clear that precipitation was skewing the clustering results and producing groups that lacked a meaningful geographic pattern that satisfied the personal requirements set out at the start of the implementation. Therefore removing the feature and focusing solely on the temperature-based features produced a much clearer and interesting climate zone, similar to what was

envisioned at the beginning of the programs implementation. This reinforced the point raised in the background theory that K-Means is quite sensitive to feature choice, which can have a direct impact on the quality of the clustering output which depends on the feature representation as much as it does on the algorithm [20][21].

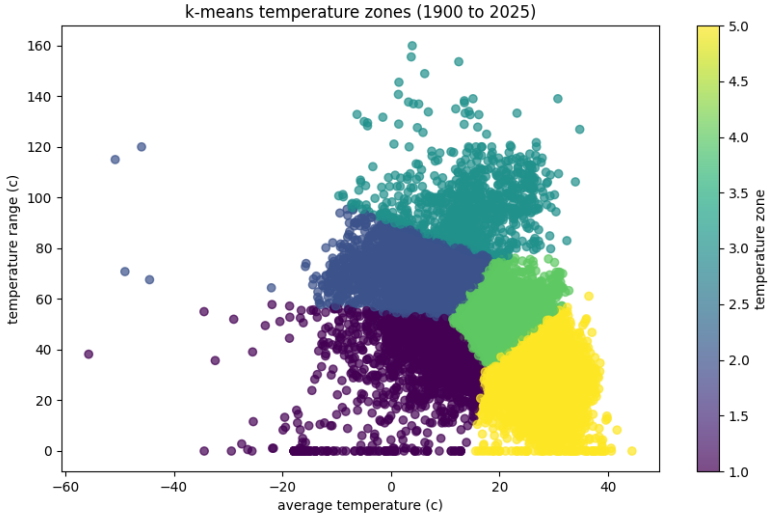


Figure 4: K-Means temperature zones scatter plot showing average temperature against temperature range.

The scatter plot above reveals that the five zones separate clearly in the feature space. Zone 1 sits in the bottom left with low temperature averages and low ranges, suggesting that the regions are where stations that are consistently cold record data. Zones 2 and 3 occupy a middle band with moderate averages but high temperature spans , which directly correlate to continental climates with a bigger seasonal variance. In the bottom-right the remaining 4 and 5 zones are with high averages and low ranges, which display characteristics typical of tropical regions where the stations would stay warm through the course of the year. The overall shape of the scatter is also notable due to the stations with very high or really low average temperatures tend to have smaller ranges, while mid-range averages show a wider spread. This aligns with the expectation that equatorial and polar stations experience most stable conditions compared to regions that are temperate.

Two visualisation scripts were developed to present the clustered results. The first was a scatter plot of average temperature against the temperature range, coloured by cluster assigned centroids, to show how the algorithm separated the stations. The second joined the clustered output with the NOAA station metadata containing longitude and latitude coordinates and plotted the results on an interactive global map using Plotly, providing a geographic interpretation of the zones created by the algorithm.

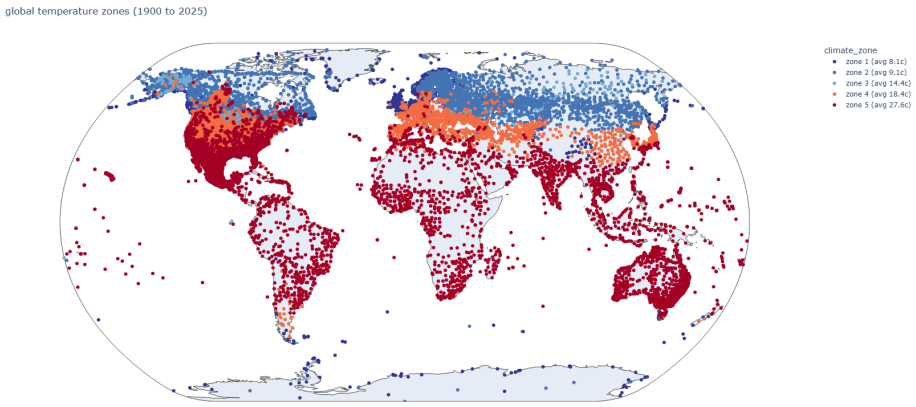


Figure 5: Global temperature zones mapped using K-Means clustering output.

The clusters were ranked from coldest to hottest based on their average temperature and labelled accordingly in both visual representations. This ensured that the zone numbering and colour scheme followed a logical temperature gradient rather than the arbitrary cluster identifiers assigned by the algorithm. As seen in the map, the five zones align with recognisable geographic climate patterns: zone 1 and 2 are covering higher latitude regions such as Scandinavia, Canada and Russia where the average temperatures are lower, zone 3 spans temperature mid-latitude areas, zone 4 captures warmer transitional regions, and zone 5 lastly corresponds to tropical and equatorial areas such as Central Africa, the South East regions of Asia and Northern Australia where average temperature are highest globally. This geographic coherence confirms that the temperature-based feature representation produced meaningful and comprehensible clusters.

4.9.1 Output Management and Post-Processing Workflow

For the later-stage consisting of numerical and machine learning workloads, a separation was enforced between the expensive cluster-side processing and the repeated visualisation and presentation stage. Rather than rerunning the entire Spark job in order to make adjustments when plotting and mapping the final output, the Spark program was initially going to have the output written in the Parquet format due to it being a highly efficient option for distributed processing and storage. However, after using Parquets for reviewing results outside the Big Data cluster environment proved to be less convenient due to the output not being directly human-readable and in an encoded format. This would have required an extra step in order to inspect and be utilised clearly.

In order to make the workflow more practical and efficient, the final clustered outputs that needed to be reviewed and processed to be visualised as scatter graphs and maps were instead exported as CSV files. Although, this format was not as efficient as Parquets for large-scale storage and computation, it had an abundance of benefits to be considered viable to use within this project. Some of the factors were that the process to manually inspect, transfer and reuse the outputs were far easier for the later visualisation stage. This avoided an unnecessary friction in the post-processing workflow, as the results could be opened directly and checked without relying on Spark.

5 Performance Analysis

5.1 Experimental Setup

Two PySpark applications - a word count and a distributed grep - were evaluated using subsets of a larger Wikipedia XML dataset ranging from 10 GB up to 90 GB and finally the full 100 GB dump. By investigating into the physical structure of the cluster being used, the CIM help desk informed that the experiments were conducted on a cluster of ten virtual machine executors hosted on a single physical server. This means all Spark tasks ultimately shared the same underlying hardware resources such as the CPUs, memory (RAM), disk and network bandwidth.

Each test was submitted with the Linux `time spark-submit` command to measure total job runtime, which was the primary metric recorded. The default HDFS block size (approx. 128 MB) was used for data partitioning. The focal point was on each jobs runtime under these controlled conditions.

5.2 Spark-submit Parameter Tuning

To gain an understanding of how each Spark configuration impacts performance, systematically varying each key parameter which includes the executor memory, CPU cores per executor and the amount of executors assigned to the Spark job. Within this section the effects of each will be presented and the test scenarios will be summarised.

5.2.1 Memory Settings

Each of the applications was run with different memory allocations to see if more memory speeds up processing. For the Distributed Grep job, the dataset was kept consistent as 100 GB of Wikipedia text, while using 6 executors with 4 cores each but varying the memory size allocated to each executor from 1 GB up to 10 GB. Similar tests were carried out on the Word Count and it showed the same pattern although it took longer to process due to the nature of the shuffle operations occurring.

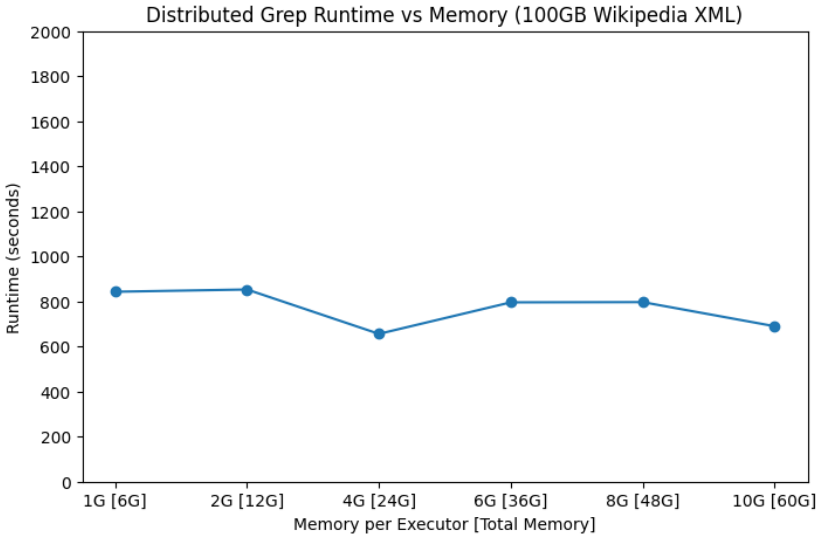


Figure 6: Distributed Grep runtime with varying executor memory.

Overall, at the range of 4 GB - 6 GB of memory allocated per executor seemed to be the point where performance seemed to increase very slightly. This would suggest that raising the amount of memory assigned to the jobs was not the main limiting factor, which correlates to the transformations taking place as the workload seems to be CPU bound due to the computation being the focal point. In summary, since the memory when allocated at around 4 GB seems to suffice and adding more memory does not provide any significant increase in runtime, their was no performance boosts to allocate more than 4 GB. Additionally, the workload did not appear to be strongly memory-bound while experimenting with this configuration, which would suggest that other cost more strongly influenced runtime like the computation, transformation overhead and in word count that would be down to the shuffle cost.

Since the Big Data server is used by many different users being responsible with the allocation of resources to a job that could take a significant amount of time to process is vital to the fair and considerate use of the cluster. This is due to the fact over-allocation should be avoided when it does not improve the runtime of a particular workload. The minor fluctuations occurring can be put down to external factors like server load or the number of concurrent jobs which could cause small variations in the recorded run times.

5.2.2 Cores and Executors

For the Word Count program's performance tuning there were a series of experimental changes involving the executors and cores per executor made on the cluster, starting of with a linear growth of just the executors to discover how the increased parallelism affected the runtime.

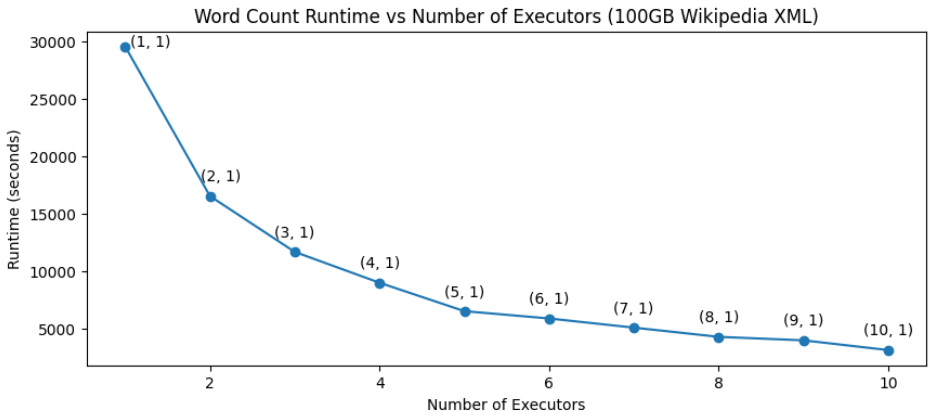


Figure 7: Word Count runtime with varying numbers of executors.

The graph displays a linear growth in the number of executors assigned to the Spark job and shows that there is a quite obvious initial performance gains via the sharp decrease in runtime when scaling from 1 to 4 executors. As more and more executors are assigned to the Spark job its impact on the runtime performance flattens significantly, which aligns with Amdahl's law, suggesting that speed-up in parallelised workloads will be increasingly limited by the sequential components of the job. In the Word Count program in particular, this is primarily a cause of the wide transformations required to group and count the words, which calls for a costly data shuffle to occur over the network. This results in a ceiling forming where the performance can no longer increase in a linear fashion and takes on an exponential form.

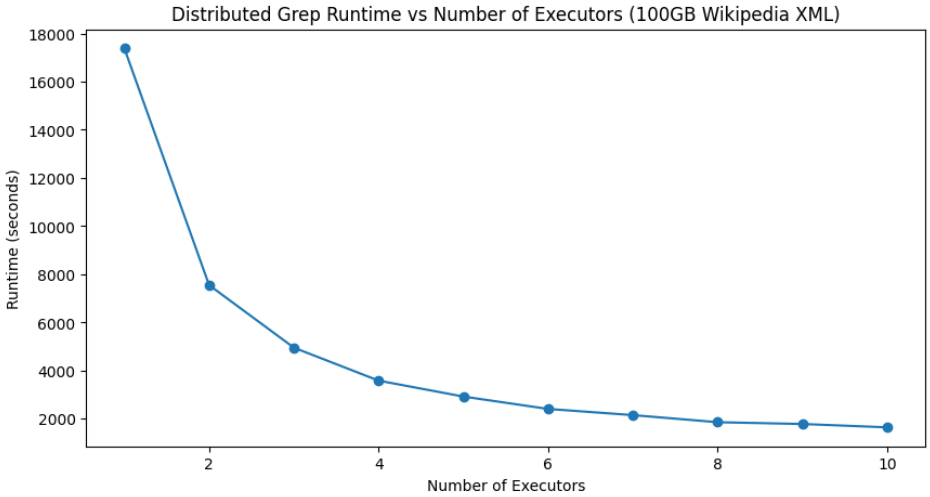


Figure 8: Distributed Grep runtime with varying numbers of executors.

The same pattern reappears when the performance testing was conducted on the Distributed Grep program, showing a much steeper decline in runtime and then the same flattening of performance. At the time, the conclusion reached was partially due to the machine potentially being bottlenecked due to its architecture being made of a single machine being split into different virtual machines rather than a standard cluster being made of multiple machines used in conjunction and forming a network used concurrently to perform computation.

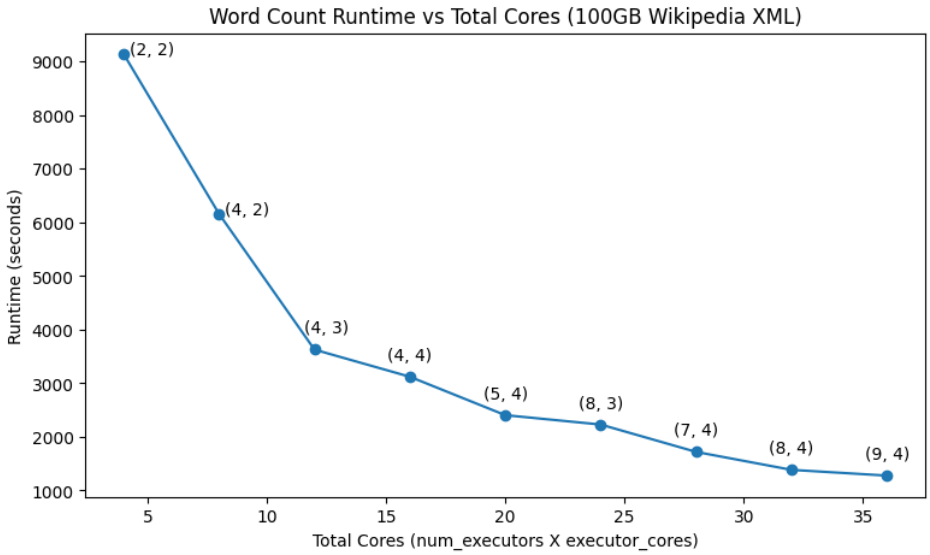


Figure 9: Word Count runtime with varying total cores.

The performance scaling across the total cores can be split into three phases, in which the first rapid improvement is seen. From 4 cores to 12 cores the rate at which run time decreases is significant falling from around 9,000 seconds to 3,600 seconds. This suggests that the cleaning and tokenisation stages of the job are benefitting from the increased parallelism, allowing the tasks within each stage to be distributed across the worker nodes. Then as seen in previous performance tests the rate of improvement slows to a

point that adding more cores provides diminishing returns, in which case those additional cores could be used to process another job.

5.3 Scalability Benchmarks

In order to evaluate the scalability of the Word Count implementation, the runtime was measured with the dataset size increasing along with the number of executors portrayed on a single graph to support the research of Amdahl's law as well as Gustafson's law which respectively are evident within the following graph.

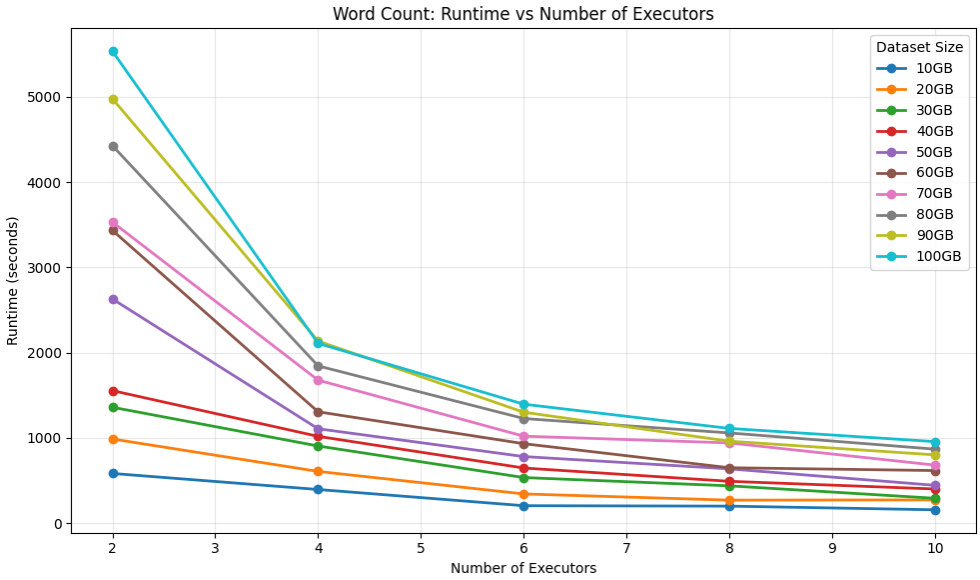


Figure 10: Word Count runtime across dataset sizes and executor counts.

The aspects of Amdahl's law present are through the gains in performance greatly decreasing as more executors are allocated due to there being limiting factors in place such as the I/O and the sequential aspects of the workload itself.

This experiment also demonstrates key aspects of Gustafson's law, which suggests that parallel systems are effective as users increase the problem size which is clearly visible due to the initial gap in performance being thinned down to a miniscule margin showing the efficiency gained by adding more resources.

Some minor fluctuations occurred as a result of the runtime data being gathered at different states of the cluster's physical architecture due to hardware issues occurring leading to increased maintenance work limiting access. This also cause failures when running extensive Spark jobs on the Cluster delaying the gathering of the relevant data needed.

5.4 Interim Recommendations

The data gathered from the 100 GB Wikipedia experiments in particular presents the optimisation that the Spark framework provides on the Big Data cluster. Based on the performance bottlenecks and results the following recommendations are to:

1. **Optimal Resource Allocation**

- **Executor and Cores:** For both the applications, the largest performance increases came from 2 executors to 6 executors. Therefore, the aim should be to use 6 executors with 4 cores each for a workload of the size used within this project. Beyond this point, there are diminishing returns due to sequential part of the job and adding more executors outweighs any benefits of the added parallelism.
- **Memory:** Increasing the executor memory over the 4 GB to 6 GB threshold gave negligible improvements and sometimes added to the runtime due to increased garbage collection overhead, meaning the pauses when clearing the unused memory became longer making the job last longer.
- **Shuffle and partitions:** For the datasets used the default HDFS block size of around 128 MB was used, for the 100 GB dataset that meant splitting the workload into 800 partitions. This impacted the runtime in which this large amount of data could be processed, as having fewer partitions would mean the stress on each individual executor would be increased, slowing down the computation.

Throughout the process of utilising the Big Data cluster, the CIM help desk provided great insight into the approach to take when testing my code. They informed me of the architecture behind the universities cluster giving a further insight into the results gathered during the performance testing phase.

5.5 Later-Phase Performance Analysis

The statistical analysis and K-Means programs were evaluated on the same weather dataset consisting of large quantities of numerical data, with the help of the Big Data cluster. Unlike the interim Wikipedia experiments which varied memory, cores and dataset sizes, the next stage testing focused on two dimensions: executor scaling and shuffle partition tuning. This decision was informed by the interim findings which showed that memory was not a main barrier for the workloads, and by the supervisor feedback which suggested varying the partition size as an additional experiment.

5.5.1 Statistical Analysis Performance

For the executor scaling test, the number of executors was increased from 2 to 10 while holding the other metrics at 4 cores and 4 GB of memory as the fixed parameters. A baseline was established with three repeated runs at 4 executors to confirm consistency, which produced the following runtimes: 862, 1211 and 878 seconds. The jump seen in the second run proved that the time of testing matters as there must have been external cluster load at the time of execution for that particular run.

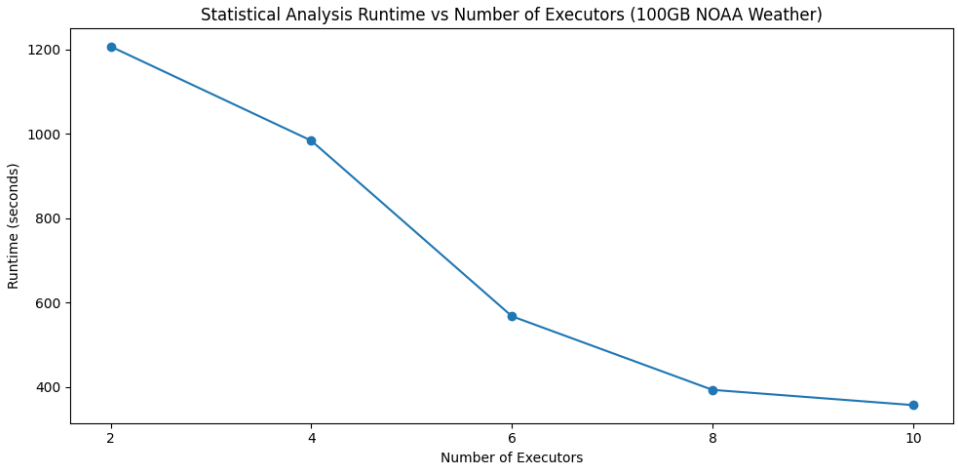


Figure 11: Statistical Analysis runtime vs number of executors.

Runtimes dropped from 1207s at 2 executors to 357s at 10 executors, with the largest improvement occurring between 4 and 6 executors where the runtime nearly halved showing the great improvement that was achieved. Throughput increased considerably from around 87 MB/s to 293 MB/s. The relative speed increase reached around 3.4x at the 10 executors relative to the 2 executor baseline, which fell below the ideal linear speedup of 5x. This gap that was created was consistent with the overhead introduced by the aggregation and shuffle stages, as seen in the description for Amdahl's law.

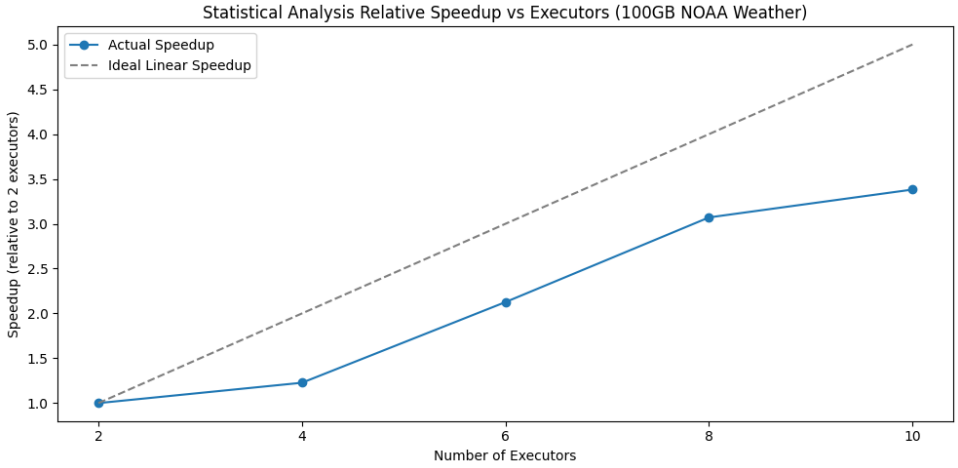


Figure 12: Statistical Analysis relative speedup vs executors.

For the partition scaling tests, the executor configuration was fixed at 4 executors with 4 cores and 4GB memory as the shuffle partition count was varied from 16 all the way up to 800. The shuffle volume remained constant at approximately 67 MB regardless of the partition count, which confirmed that the statistical analysis workload involved a fixed amount of data exchange during the aggregation stage. The lowest runtime was seen at 128 partitions resulting in the time, 884 seconds, while the time increased at 800 partitions, which was the culmination of added overhead due to the cost of managing a larger number of heavily divided tasks. One run at 400 produced a significantly lower runtime of 627 seconds which was not replicable in the two additional runs at the same configuration suggesting it was an anomaly rather than a noteworthy result.

5.5.2 K-Means Performance

The K-Means executor scaling tests followed the same configuration as the statistical analysis, increasing from 2 to 10 executors once more. Runtime dropped from 759 seconds to 256 seconds, with throughput rising from 138 MB/s to 409 MB/s. The K-Means program achieved high absolute throughput than the statistical analysis program at the same configuration settings, which may appear counterintuitive given that K-Means is the more computationally complex workload. This is quickly understood as a misconception as the K-Means pipeline benefits from the feature extraction stage reducing the dataset from the hundreds of millions of raw records down to a smaller subset before the iterative clustering process commencing. This meant that the expensive stages operate on a smaller volume of data specific to the feature set required for the meaningful data visualisation that was achieved with the global mapping stage.

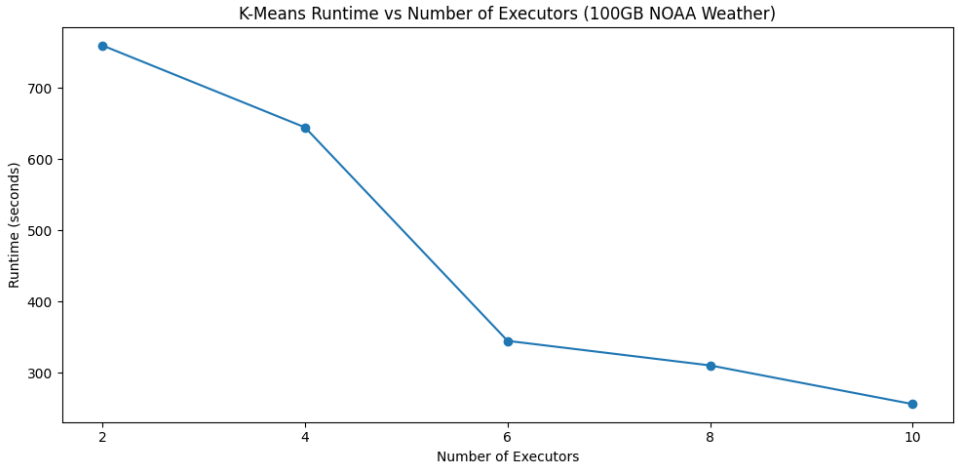


Figure 13: K-Means runtime vs number of executors.

The partition scaling tests revealed a key difference between the workloads. Unlike the statistical analysis where the shuffle volume stayed at a consistent rate of 67 MB/s, the K-Means shuffle volume grew from around 1 GB at 16 partitions to 1.8 GB at 1200 partitions. This is a direct consequence of the iterative nature that the algorithm possesses, where data is redistributed across partitions during each centroid recalculation and assignment step. The lowest runtime of 421 seconds was reached at 800 partitions, but the increase further to 1200 partitions caused the runtime to rise to 678 seconds as the additional shuffle overhead outweighed the benefit that comes with finer parallel activation.

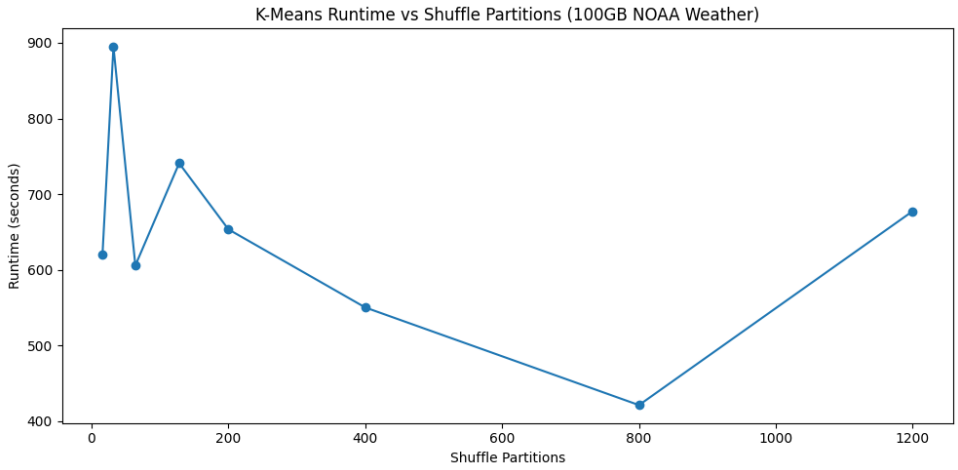


Figure 14: K-Means runtime vs shuffle partitions.

5.6 Combined Recommendations

Bringing together the findings from the programs, the following configuration recommendations apply to workloads of similar scale on this cluster. Using 6 to 8 executors, each with 4 cores which provides the best balance between parallelism across the workload types. Memory of 4 GB per executor is sufficient as no significant improvements are observed beyond this point, as well as the fact that other user's workloads would be running and requesting more memory than needed would go against hygienic cluster usage. For aggregation workloads like the statistical analysis, the standard partition count or a moderate value around 128 to 200 is appropriate since the shuffle volume is fixed. Whereas, for iterative machine learning workloads like the K-Means algorithm, where partition tuning is more sensitive and a value in the range 400 to 800 provided the most optimal results, balancing parallelism again the growing shuffle overhead.

6 Critical Analysis and Discussion

This section will reflect on the outcomes brought about by the project, comparing the behaviour of the programs implemented, evaluating the quality of the results produced and discussing any limitations encountered during the project.

6.1 Comparing Workload Types

The four programs implemented across the project exposed some different distributed computing behaviours. The Word Count program was the most shuffle-heavy workload due to the global aggregation required to count every unique word across the massive dataset. This resulted in the longest runtimes when scaling as well as displaying the most pronounced diminishing returns when scaling executors. The Distributed Grep program was primarily a filtering workload with no global aggregation, meaning it scaled more steeply with added parallelism and an faster than the Word Count with the same resources via the configuration of the cluster.

The later-phase programs on the NOAA weather dataset introduced a few different workload profiles. The first program, statistical analysis, combined narrow transformations for cleaning with a single aggregation step for the summary statistics and a Bucketizer pass for the frequency distribution. This produced a compact DAG with minimal shuffle overhead. The K-Means program was the most computationally complex workload due to its iterative nature seen by the number of stages involved within the program, which required repeated shuffles for centroid recalculations and assignment across multiple iterations. This resulted in shuffle volumes that scaled with partition count, unlike the statistical analysis where shuffle remained constant even when the partition metric was changed.

These contrasting outcomes demonstrate that the optimal Spark configuration is workload-dependent and is not as simple as selecting a few metrics and assigning them to all types of diverse workloads. A filtering workload like Distributed Grep would benefit the most from raw parallelism being accounted for, where as a shuffle heavy workload like Word Count reaches diminishing returns earlier as the sequential synchronisation hurdle bought about by the shuffle stage. The final programs show that the aggregation workloads and iterative machine learning workload each have their own tuning features, reiterating the importance of understanding the computation of a job that is taking place before it is allocated any cluster resources.

6.2 Quality of Results

The clustering output from the K-Means program produced a geographically coherent temperature zones that aligned with recognisable patterns of global norms, which created confidence that the feature representation and algorithm configuration were appropriate for the dataset. The statistical analysis outputs were validated against the unit test made during development, and the frequency distribution followed an expected shape for global maximum temperature recording with the majority of observations falling in the moderate range.

The performance testing results were broadly consistent with the theoretical expectations set out in the background theory section. The diminishing returns that were observed when scaling executors aligned with Amdahl's law, and the ability to process

larger dataset more efficiently with additional cluster resources supported the principles described by Gustafson's law. The relative speedup graphs showed that actual scaling fell below the ideal linear range for the workloads, which is expected give the sequential components that are present in all the programs.

6.3 Limitations

Several limitations should be acknowledged as in the university cluster it consisted of ten virtual machines hosted on a single physical server, which would mean that the network communication between the executors did not reflect the behaviour of a true multi-node cluster where network latency and bandwidth would be more prominent factors, which could mask bottlenecks that would appear in a production distributed environment.

The performance tests were conducted on a shared cluster where other users may have been running their heavy workloads and job concurrently potentially skewing outcomes. Although efforts were made to repeat runs and record consistent results some fluctuations in the recorded runtimes are likely attributed to external server load rather than the configurations that were being tested. Hardware maintenance issues also caused delays and limited access to the cluster during parts of the project.

7 Software Engineering

7.1 Methodology

7.1.1 Iterative Development

The project was initiated by gaining an understanding of the defined scope to be achieved in the early deliverables of the project description: implementing two Spark applications being the Word Count and Distributed Grep programs capable of processing a 100 GB Large Wikipedia Dataset.

An Iterative Development Approach was used in the project by developing and testing the Spark jobs on smaller, more manageable subsets of the larger dataset before being able to scale up. This approach of prototyping before a full scale implementation allowed me to debug any issues and gradually refine the output of the cleaned text and final outputs to validate the intended results. This saved a great deal of time as having to utilise the large dataset would be the antithesis of an optimal development strategy.

The GitLab repository was used as a version control system to manage the code and ensure segmentation between stages of development. Frequent, descriptive commits added to the evolution of the repository, along with issues which related the commits to an ultimate goal that was described showing the required actions to achieve the outcome and what the expected deliverables would be as the task set out by the issue was completed. Branches were used to partition the working code in main from the code in development for the Word Count program as an example to make sure the main codebase remained clean. A Dairy file tracking the regular progress was maintained to record any decisions, challenges and key concepts to focus on, as well as any supervisor meetings, through the course of building the project.

7.1.2 Testing Methodology

For the latter half of the project, a test-driven development (TDD) approach was adopted for the programs to validate the statistical analysis and clustering logic before running them at scale of the Big Data cluster. This was a deliberate shift from the interim phase, where correctness was verified manually by inspecting smaller output samples locally, then implementing a small version over the cluster and finally a "Big Data" implementation using the full dataset. The decision to introduce formal unit testing was to reinforce the logic worked and would not have to be checked over the real data.

A key challenge with testing PySpark code is that unit tests need to be fast as well as deterministic, which rules out the use of the real data in CSV files. After researching PySpark testing practises, the approach taken was to create a local Spark session within the test fixtures and construct small mock DataFrames with known values. This would allow each function to be tested in isolation against expected outputs without requiring running them over the cluster.

For the statistical analysis module, pytest was used as the test framework. A single local Spark session was created that was shared across the test functions and stopped after the test suite completed. Three test cases were written: one to verify that the summary statistics function returned the correct count, minimum, maximum and median for a known set of temperature records; the next was to confirm that the frequency distribution preserved the total row count across all intervals; and the other was to check that the number of split points and bucket counts matched the requested amount of intervals.

PySpark's `assertDataFrameEqual` function was also used to compare those distributed DataFrames directly, which created a more robust and reliable comparison than collecting results to the driver and checking those values manually.

The K-Means module used the unittest test framework. The `TestClimateClusterer` class followed a setup and teardown pattern where a local Spark session and an instance of the `ClimateClusterer` class were mode once before all tests and stopped once completed. Three tests were written covering each stage of the pipeline: `test_clean_data`, which verified that the cleaning function correctly filtered out any unnecessary values and converted them into the required standard to keep the feature set clean; `test_extract_features`, which checked that the feature aggregation produced the correct average, maximum, minimum and temperature range for a few mock stations with known values; lastly `test_applykmeans`, which confirmed that the clustering step produced a `cluster_id` column and retained all the inputted stations.

7.1.3 Object-Oriented Design

The later stage of programs also introduced a shift towards an object-oriented structure that was not established during the interim phase of development. The Word Count and Distributed Grep programs were both written as procedural scripts, which was appropriate for their straightforward pipeline structure. Although, as the K-Means program became more complex and due to procedural programs being worse to unit test, it became clear that encapsulating the logic within a class containing functions would prove better for organisation and reusability of the individual functions through the course of the project.

The `ClimateClusterer` class was designed to hold the Spark session, the NOAA schema and methods for each pipeline stage. This allowed the same class to be instantiated with a local Spark session for testing or with the cluster session for the full-scale run without changing the core logic. The constructor accepted an optional `spark_session` parameter, meaning the test suite could inject its own local session while the Big Data cluster script could pass in the session created by the `spark-submit` environment.

This separation also enabled the local visualisation scripts to import the `ClimateClusterer` class directly, avoiding having to reuse code across files and between local test runs and the cluster execution. The project directory was structured with a testing folder and source folder within an overseeing Testing folder for local runs, the files inside contained reusable modules. This way the production code was separated from the various test code files.

7.2 System Design

The Word Count program's flow is depicted by the following UML diagram:

This UML is a high level depiction of how the Word Count program's pipeline ran in a more linear view. This process in reality is split across the cluster and the partitions of the large dataset, with each worker node within the cluster contributing towards the overall computation.

The Distributed Grep program follows a similar design shown in the UML diagram below:

The flow is now split into three sections being the data source, meaning the HDFS storage system on the Big Data cluster, which then leads to the computations conducted

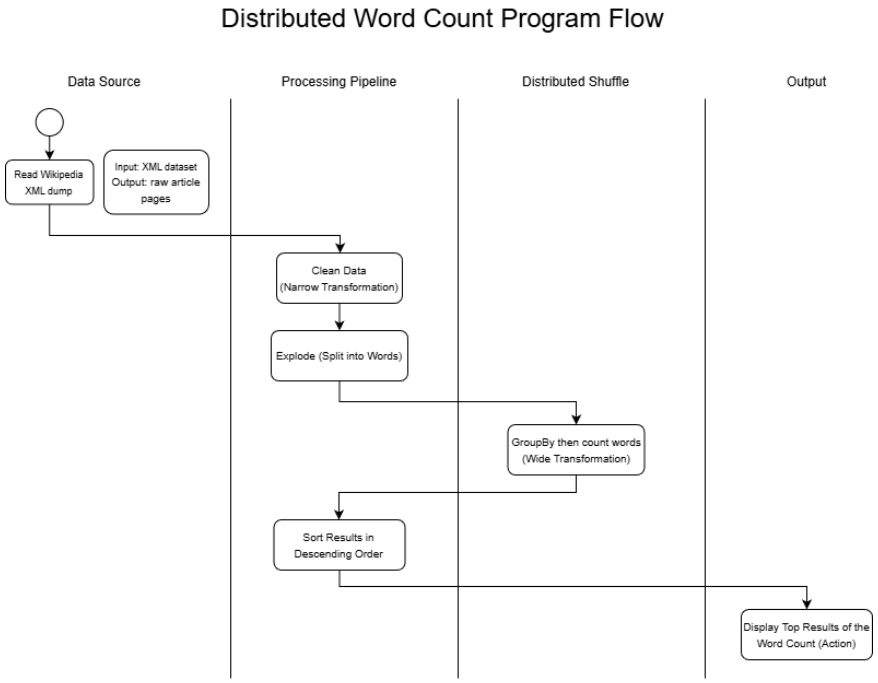


Figure 15: High-level flow of the Distributed Word Count program.

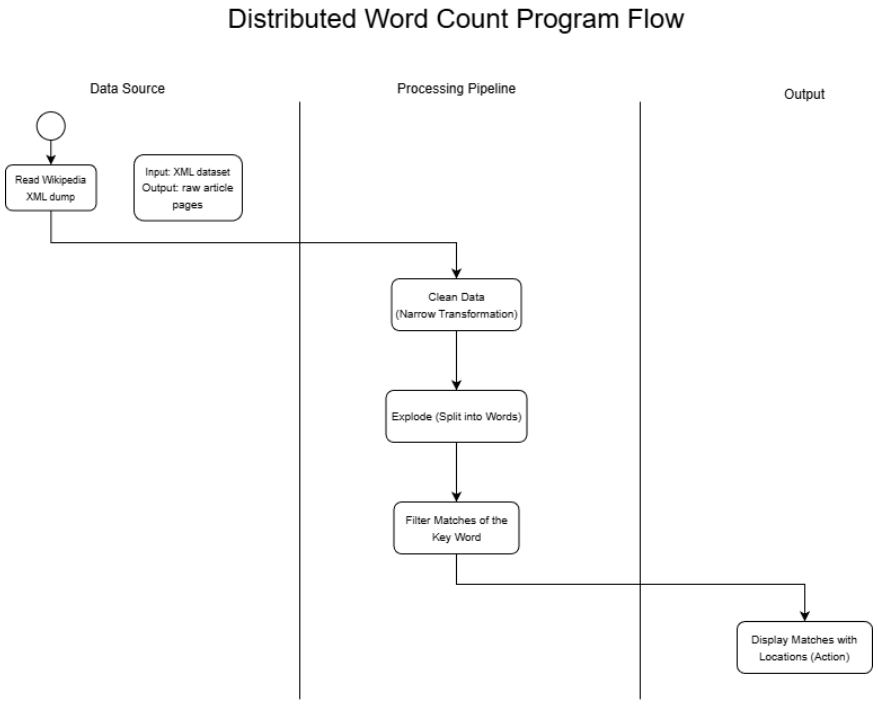


Figure 16: High-level flow of the Distributed Grep program.

on the data partitions and finally outputs some matches as a way to verify the program has completed the processing and displayed the results correctly.

References

[1] Bartley, K. (2025) 'Data Statistics (2025) - How much data is there in the world?', Rivery, 28 May. Available at: https://rivery.io/blog/big-data-statistics-how-much-data-is-there-in-the-world/ (Accessed: 27 November 2025).

[2] Jules S. Damji, Brooke Wenig, Tathagata Das, Denny Lee (2020) *Learning Spark: Lightning-Fast Data Analytics*, 2nd Edition, O'Reilly Media. (Accessed: 10 October 2025).

[3] Apache Hadoop: Advantages, Disadvantages, and Alternatives (2022) AltexSoft. Available at: https://www.altexsoft.com/blog/hadoop-pros-cons/ (Accessed: 5 October 2025).

[4] What is Big Data? — IBM (2024). Available at: `https://www.ibm.com/think/topics/big-data` (Accessed: 3 October 2025).

[5] Tang, N. (2014) 'Big Data Cleaning', in L. Chen et al. (eds) Web Technologies and Applications. Cham: Springer International Publishing, pp. 13–24. Available at: `https://doi.org/10.1007/978-3-319-11116-2_2` (Accessed: 20 November 2025).

[6] Dean, J. and Ghemawat, S. (2008) 'MapReduce: simplified data processing on large clusters', Communications of the ACM, 51(1), pp. 107–113. Available at: https://doi.org/10.1145/1327452.1327492 (Accessed: 21 November 2025).

[7] Benlachmi, Y. and Hasnaoui, M.L. (2020) 'Big data and Spark: Comparison with Hadoop', in 2020 Fourth World Conference on Smart Trends in Systems, Security and Sustainability (WorldS4). 2020 Fourth World Conference on Smart Trends in Systems, Security and Sustainability (WorldS4), pp. 811–817. Available at: https://doi.org/10.1109/WorldS450073.2020.9210353 (Accessed: 21 November 2025).

[8] Xiao, W. and Hu, J. (2020) 'A Survey of Parallel Clustering Algorithms Based on Spark', Scientific Programming, 2020(1), p. 8884926. Available at: `https://doi.org/10.1155/2020/8884926` (Accessed: 27 November 2025).

[9] Rajeh, W. (2022) 'Hadoop Distributed File System Security Challenges and Examination of Unauthorized Access Issue', Journal of Information Security, 13(2), pp. 23–42. Available at: https://doi.org/10.4236/jis.2022.132002 (Accessed: 25 October 2025).

[10] Zaharia, M. et al. (2010) 'Spark: Cluster Computing with Working Sets'. Available at: `https://www.usenix.org/legacy/event/hotcloud10/tech/full_papers/Zaharia.pdf` (Accessed: 29th October 2025).

[11] Manconi, A. et al. (2023) 'Framing Apache Spark in life sciences', Heliyon, 9(2), p. e13368. Available at: https://doi.org/10.1016/j.heliyon.2023.e13368. (Accessed: 15 November 2025).

[12] Zaharia, M. et al. (2012) 'Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing'. (Accessed: 27th November).

[13] Deep Dive into Spark SQL's Catalyst Optimizer (2015) Databricks. Available at: https://www.databricks.com/blog/2015/04/13/deep-dive-into-spark-sqls-catalyst-optimizer.html (Accessed: 2nd December 2025).

[14] 1. Introduction to Parallel Computing (2003). Available at: https://learning.oreilly.com/library/view/introduction-to-parallel/0201648652/ch01.html (Accessed: 13 November 2025).

[15] Hill, M.D. and Marty, M.R. (2008) 'Amdahl's Law in the Multicore Era', Computer, 41(7), pp. 33–38. Available at: https://doi.org/10.1109/MC.2008.209.

[16] Ahmed, N. et al. (2021) 'A parallelization model for performance characterization of Spark Big Data jobs on Hadoop clusters', Journal of Big Data, 8(1), p. 107. Available at: https://doi.org/10.1186/s40537-021-00499-7.

[17] Greenwald, M. and Khanna, S. (2001) 'Space-Efficient Online Computation of Quantile Summaries', in *Proceedings of the 2001 ACM SIGMOD International Conference on Management of Data*. New York: ACM, pp. 58–66. Available at: https://doi.org/10.1145/375663.375670

[18] PySpark DataFrameStatFunctions.approxQuantile Documentation. Apache Software Foundation. Available at: https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrameStatFunctions.approxQuantile.html

[19] PySpark MLlib Bucketizer Documentation. Apache Software Foundation. Available at: https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.ml.feature.Bucketizer.html

[20] PySpark MLlib KMeans Documentation. Apache Software Foundation. Available at: https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.ml.clustering.KMeans.html

[21] Bahmani, B. et al. (2012) 'Scalable K-Means++', Proceedings of the VLDB Endowment, 5(7), pp. 622–633. Available at: https://vldb.org/pvldb/vol5/p622_bahmanbahmani_vldb2012.pdf

[22] Meng, X. et al. (2016) 'MLlib: Machine Learning in Apache Spark', Journal of Machine Learning Research, 17(34), pp. 1–7. Available at: https://www.jmlr.org/papers/volume17/15-237/15-237.pdf

[23] PySpark ClusteringEvaluator Documentation. Apache Software Foundation. Available at: https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.ml.evaluation.ClusteringEvaluator.html

[24] Abadi, D., Boncz, P. and Harizopoulos, S. (2013) 'The Design and Implementation of Modern Column-Oriented Database Systems', Foundations and Trends in Databases, 5(3), pp. 197–280. Available at: https://doi.org/10.1561/1900000024

[25] Melnik, S. et al. (2010) 'Dremel: Interactive Analysis of Web-Scale Datasets', Proceedings of the VLDB Endowment, 3(1), pp. 330–339. Available at: https://doi.org/10.14778/1920841.1920886